Exercise 6: Multivariate Optimization 1

Optimization in Machine Learning

> **Reproducibility**
>
> Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.
>
> **Python**
>
> ```
> pip install numpy==2.0.2 matplotlib==3.10.8
> ```
>
> **R**
>
> ```
> install.packages("remotes")
> remotes::install_version("ggplot2", "3.5.1")
> remotes::install_version("gridExtra", "2.3")
> ```

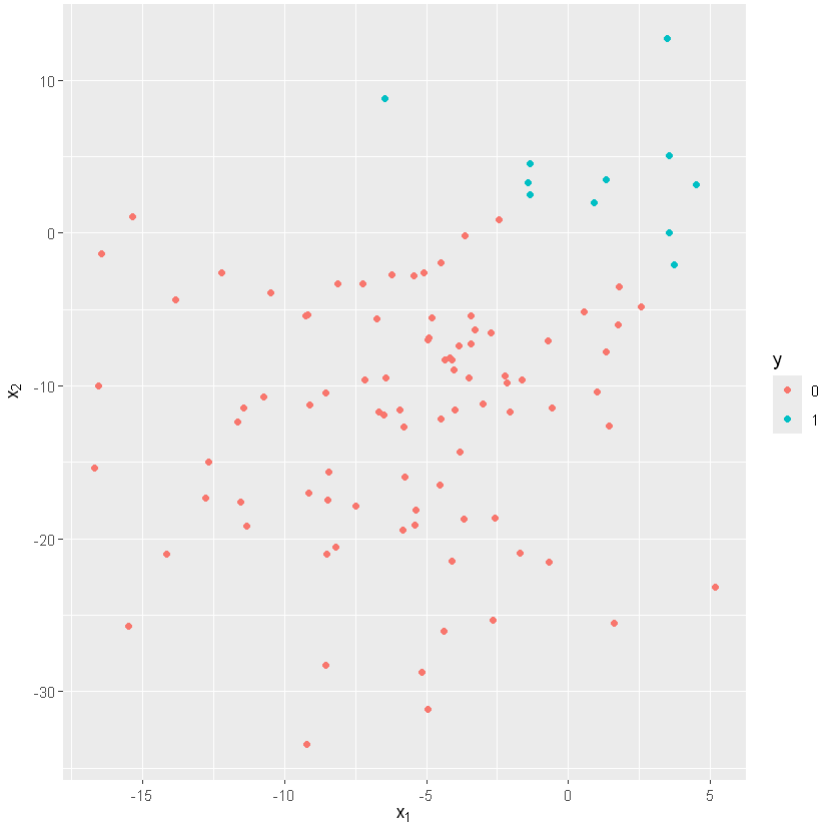
Setup

You are given the following data situation:

R

```
library(ggplot2)
library(gridExtra)

set.seed(314)
n <- 100
X <- cbind(rnorm(n, -5, 5), rnorm(n, -10, 10))
z <- 2 * X[, 1] + 3 * X[, 2]
```

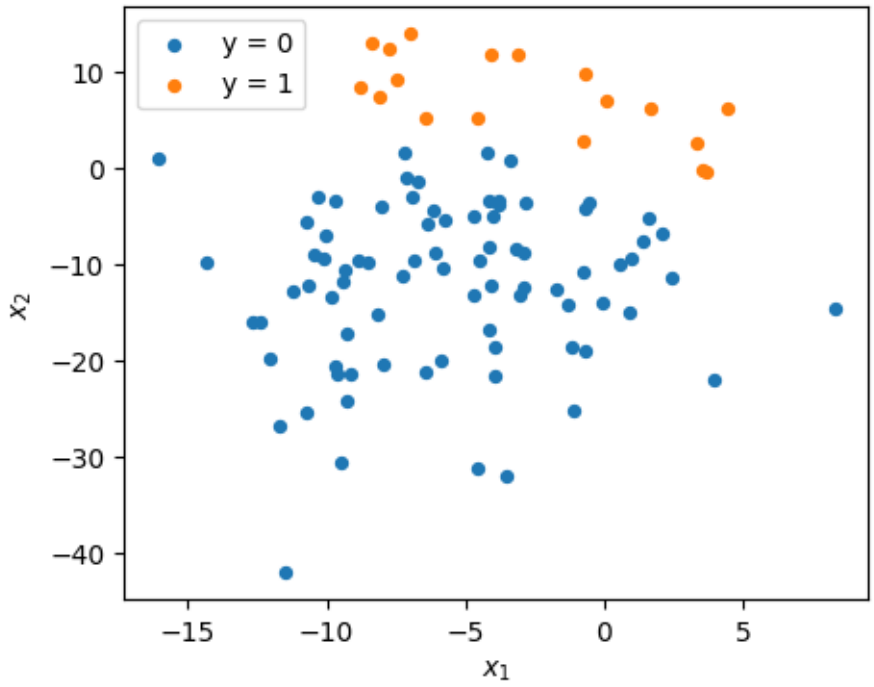
```r
pr <- 1 / (1 + exp(-z))
y <- as.integer(pr > 0.5)
df <- data.frame(X = X, y = y)

ggplot(df) +
  geom_point(aes(x = X.1, y = X.2, color = as.factor(y))) +
  xlab(expression(x[1])) +
  ylab(expression(x[2])) +
  labs(colour = "y")
```



Python

```python
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(314)
```

```python
n = 100
X = np.column_stack([np.random.normal(-5, 5, n), np.random.normal(-10, 10, n)])
z = 2 * X[:, 0] + 3 * X[:, 1]
pr = 1 / (1 + np.exp(-z))
y = (pr > 0.5).astype(int)

fig, ax = plt.subplots(figsize=(5, 4))
for label in (0, 1):
    mask = y == label
    ax.scatter(X[mask, 0], X[mask, 1], label=f"y = {label}", s=18)
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
ax.legend()
plt.show()
```



Note

R vs. Python parity. R's `set.seed(314)` and Python's `np.random.seed(314)` use *different* RNGs. The data above (and consequently the GD trajectories below) differ between languages. Both implementations are correct; only the underlying data differs. Compare *algorithmic behavior* (divergence, convergence, backtracking adaptation) rather than expecting numerically identical θ values.

In the following we want to estimate a logistic regression *without intercept* via gradient descent.

Note: We use vanilla GD for educational purposes; in practice more sophisticated algorithms are typically preferred.

(a) Complete separation makes the empirical risk unbounded below

The data above is **completely separable**: the two classes can be perfectly classified with a linear classifier. Show that in this situation, if $\tilde{\theta}$ perfectly classifies the data then

$$\mathcal{R}_{\text{emp}}(\tilde{\theta}) > \mathcal{R}_{\text{emp}}(\alpha\tilde{\theta}) \quad \text{for all } \alpha > 1.$$

Implication: the per-observation log-loss can be driven arbitrarily low by scaling θ, so the empirical risk has no finite minimum.

Solution.

We start with

$$\mathcal{R}_{\text{emp}}(\tilde{\theta}) = \sum_{i=1}^{n} \log\Big(1 + \exp(\tilde{\theta}^\top \mathbf{x}^{(i)})\Big) - y^{(i)} \tilde{\theta}^\top \mathbf{x}^{(i)} = \sum_{i=1}^{n} \begin{cases} \log\Big(1 + \exp(\tilde{\theta}^\top \mathbf{x}^{(i)})\Big) & \text{if } y^{(i)} = 0, \\ \log\Big(1 + \exp(\tilde{\theta}^\top \mathbf{x}^{(i)})\Big) - \tilde{\theta}^\top \mathbf{x}^{(i)} & \text{if } y^{(i)} = 1. \end{cases}$$

Since $\tilde{\theta}$ perfectly classifies the data, we know that

$$\begin{cases} \tilde{\theta}^\top \mathbf{x}^{(i)} < 0 & \text{if } y^{(i)} = 0, \\ \tilde{\theta}^\top \mathbf{x}^{(i)} \geq 0 & \text{if } y^{(i)} = 1. \end{cases}$$

Hence, we can focus on the functions $g(z) = \log(1 + \exp(-z))$ and $h(z) = \log(1 + \exp(z)) - z$ for $z > 0$ and study their monotonicity. (The $y = 1$ term is $h(z)$ with $z \geq 0$ directly; the $y = 0$ term $\log(1 + \exp(z))$ with $z < 0$ is $g(-z)$ with $-z > 0$. As functions g and h coincide — $\log(1 + \exp(z)) - z = \log(1 + \exp(-z))$ — so the two names just track which label case we are in.) We compute

$$g'(z) = -\underbrace{\frac{\exp(-z)}{1 + \exp(-z)}}_{>0} < 0, \qquad h'(z) = \underbrace{\frac{\exp(z)}{1 + \exp(z)}}_{<1} - 1 < 0.$$

Therefore, both g and h are strictly monotonically decreasing. It follows that $\mathcal{R}_{\text{emp}}(\tilde{\theta}) > \mathcal{R}_{\text{emp}}(\alpha\tilde{\theta})$ for $\alpha > 1$.

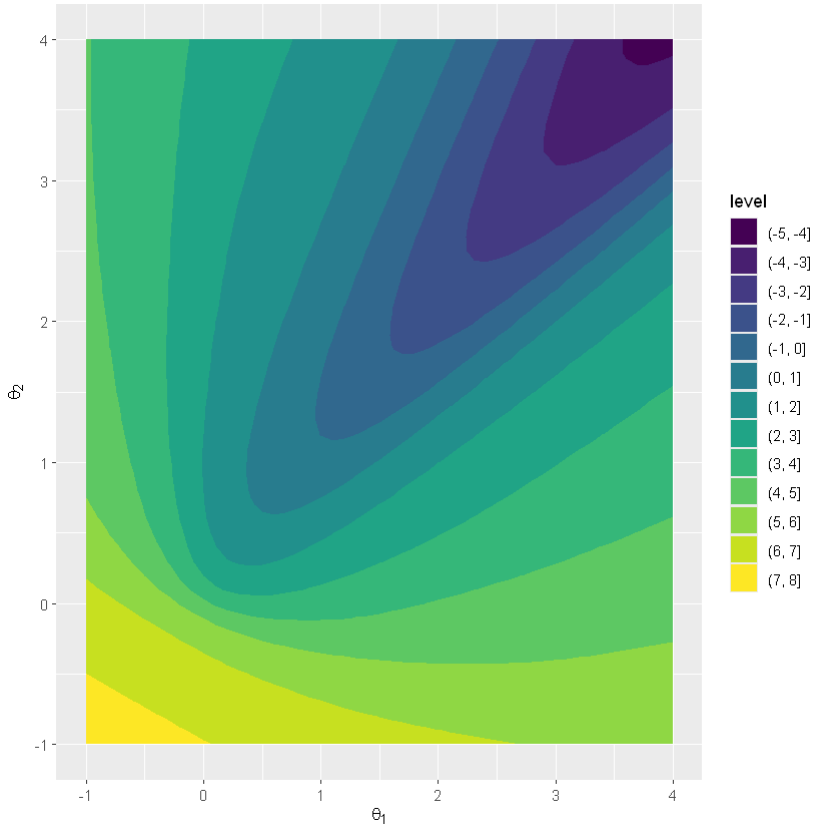
(b) Visualize the empirical risk

Visualize $\mathcal{R}_{\text{emp}}$ on $[-1, 4] \times [-1, 4]$.

Solution.

R

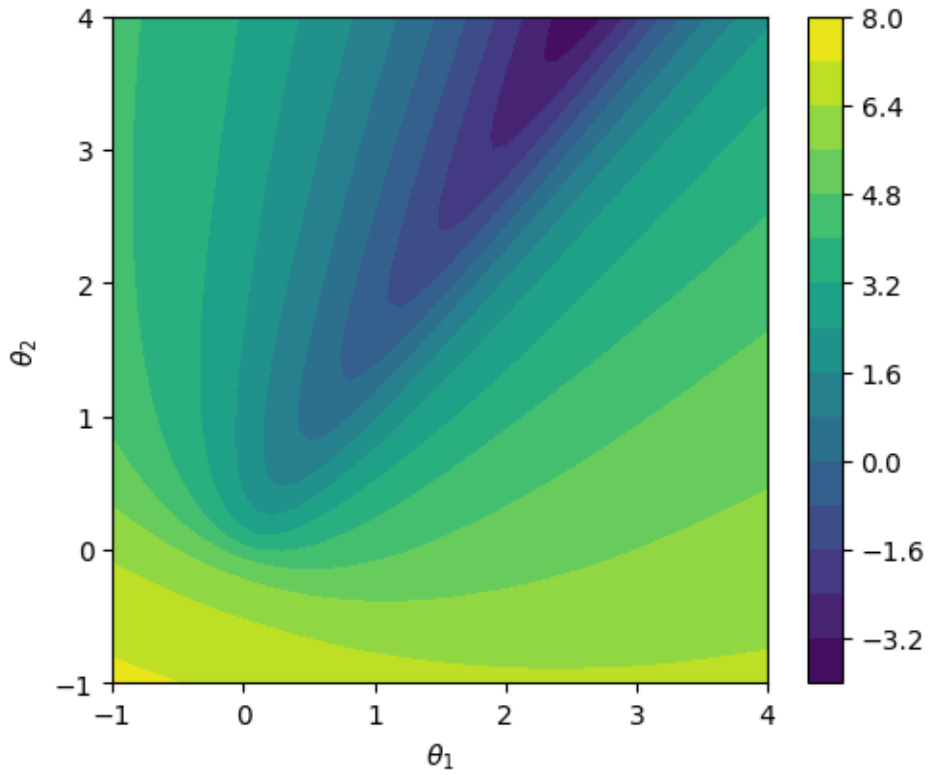
```r
# Empirical risk (lambda = 0 -> unregularized)
f <- function(theta, lambda) {
  z <- X %*% theta
  lambda * (theta %*% theta) + sum(-y * z + log(1 + exp(z)))
}

x <- seq(-1, 4, by = 0.1)
xx <- expand.grid(X1 = x, X2 = x)
# log-scale (as in (f)) to reveal the level-set shape across the wide range
fxx <- log(apply(xx, 1, function(t) f(t, 0)))
df_grid <- data.frame(xx, fxx = fxx)

ggplot(df_grid, aes(x = X1, y = X2, z = fxx)) +
  geom_contour_filled() +
  xlab(expression(theta[1])) +
  ylab(expression(theta[2]))
```



Python

```python
def f(theta, lam):
    """L2-regularized logistic empirical risk (lam = 0 -> unregularized)."""
    theta = np.asarray(theta, dtype=float)
    z = X @ theta
    return float(lam * (theta @ theta) + np.sum(-y * z + np.log(1 + np.exp(z))))


grid = np.arange(-1, 4 + 0.1, 0.1)
T1, T2 = np.meshgrid(grid, grid)
# log-scale (as in (f)) to reveal the level-set shape across the wide range
Z = np.log(np.array([[f(np.array([t1, t2]), 0) for t1 in grid] for t2 in grid]))

fig, ax = plt.subplots(figsize=(5.5, 4.5))
cs = ax.contourf(T1, T2, Z, levels=14)
ax.set_xlabel(r"$\theta_1$"); ax.set_ylabel(r"$\theta_2$")
```

```
fig.colorbar(cs)
plt.show()
```



The level sets of $\mathcal{R}_{\text{emp}}$ open toward the upper-right: the risk decreases without bound as θ moves along that direction, consistent with (a).

(c) Gradient of the empirical risk

Find $\nabla_\theta \mathcal{R}_{\text{emp}}$ for arbitrary θ.

Solution.

Differentiating term-by-term:

$$\nabla_\theta \mathcal{R}_{\text{emp}} = \sum_{i=1}^{n} \left(\sigma(\theta^\top \mathbf{x}^{(i)}) - y^{(i)}\right) \mathbf{x}^{(i)}, \qquad \sigma(z) = \frac{1}{1+e^{-z}}.$$

Equivalently in vector form: $\mathbf{X}^\top(\sigma(\mathbf{X}\theta) - \mathbf{y})$ where σ is applied element-wise. This is the standard logistic-regression gradient.

Note on regularization. When the L2-regularized objective is $\lambda\|\theta\|^2 + \mathcal{R}_{\text{emp}}(\theta)$, the regularizer contributes an extra $2\lambda\theta$ to the gradient — important for parts (e), (g).

(d) Vanilla gradient descent

Solve the logistic regression via gradient descent. Use step size $\alpha = 0.01$, starting point $\theta^{[0]} = (0, 0)^\top$, and train for 500 steps. Repeat with $\alpha = 0.02$. Explain what you observe.

Hint: recall (a).

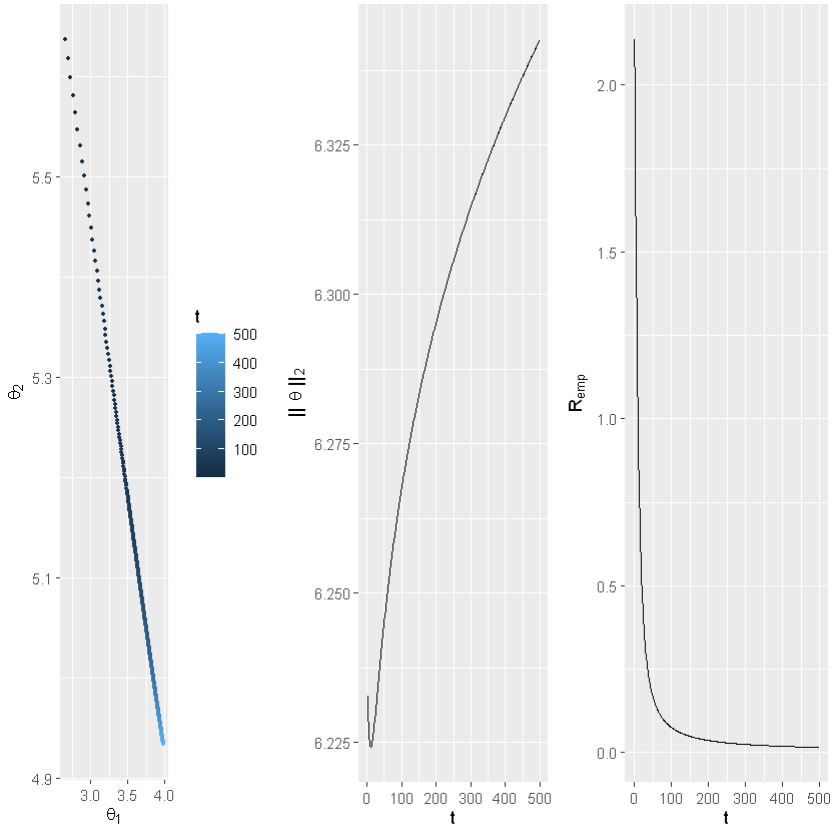
Solution.

R

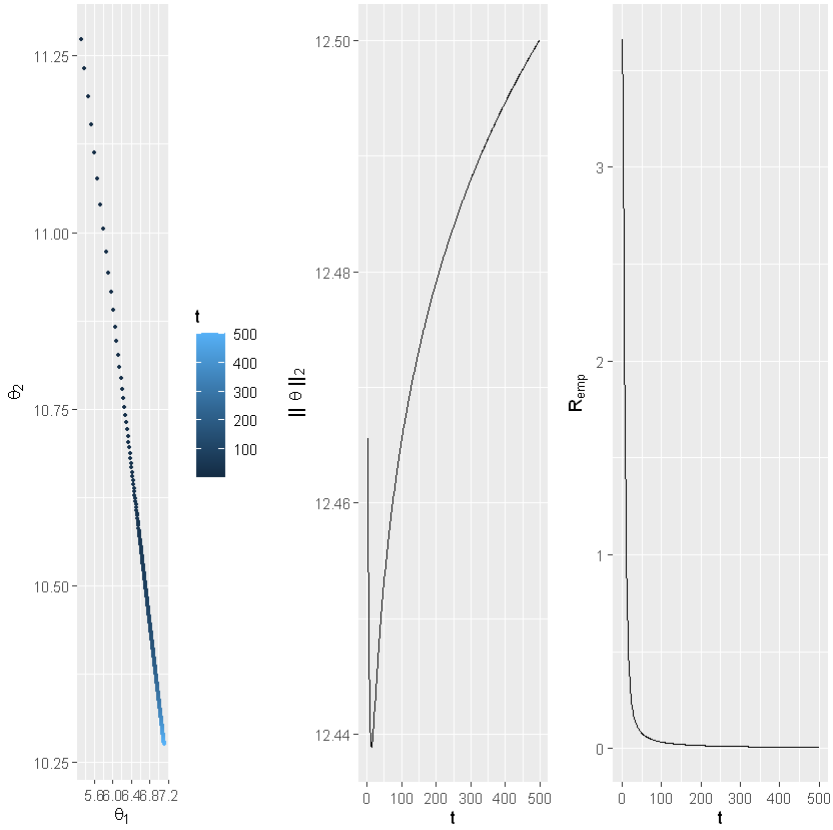
```
# Gradient of the (regularized) logistic empirical risk.
# Returns a 1x2 row vector. NOTE: the regularizer contributes 2*lambda*theta.
df_t <- function(theta, lambda) {
  2 * lambda * t(theta) - (t(y) %*% X) +
    t(1 / (1 + exp(-X %*% theta))) %*% X
}

gd_step <- function(theta, alpha, lambda) {
  theta - alpha * df_t(theta, lambda)[1, ]
}

# Run GD for n_iters; show trace, ||theta||, R_emp; print + return final state.
plot_fun <- function(step_fn, lambda, n_iters = 500, label = "") {
  theta <- c(0, 0)
  thetas <- matrix(NA, n_iters, 2)
  norms <- numeric(n_iters)
  fs <- numeric(n_iters)
  for (i in 1:n_iters) {
    theta <- step_fn(theta)        # one GD-style update
    thetas[i, ] <- theta
    norms[i] <- sqrt(sum(theta^2))
    fs[i] <- f(theta, lambda)
  }
  df_trace <- data.frame(thetas, t = 1:n_iters)
  trace_plot <- ggplot(df_trace, aes(x = X1, y = X2, colour = t)) +
    geom_point(size = 0.7) +
    xlab(expression(theta[1])) + ylab(expression(theta[2]))
```

```
  norm_plot <- ggplot(data.frame(t = 1:n_iters, n = norms), aes(t, n)) +
    geom_line() + ylab(expression("||" ~ theta ~ "||"[2]))
  remp_plot <- ggplot(data.frame(t = 1:n_iters, fs = fs), aes(t, fs)) +
    geom_line() + ylab(expression(R[emp]))
  grid.arrange(trace_plot, norm_plot, remp_plot, ncol = 3)
  cat(sprintf(paste0(
    "%-22s final theta = (%7.4f, %7.4f), ",
    "||theta|| = %7.3f, R_emp = %.3f\n"),
              label, thetas[n_iters, 1], thetas[n_iters, 2],
              norms[n_iters], fs[n_iters]))
  invisible(list(thetas = thetas, norms = norms, fs = fs))
}

## alpha = 0.01, lambda = 0 (vanilla GD, unregularized)
plot_fun(function(theta) gd_step(theta, 0.01, 0), lambda = 0,
         label = "vanilla a=0.01")
## alpha = 0.02, lambda = 0
plot_fun(function(theta) gd_step(theta, 0.02, 0), lambda = 0,
         label = "vanilla a=0.02")
```

```
vanilla a=0.01         final theta = ( 3.9867,  4.9332), ||theta|| =   6.343, R_emp = 0.015
```



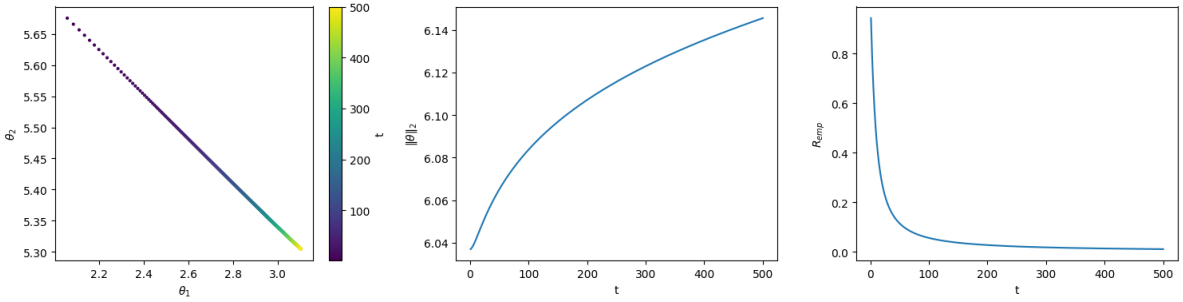
vanilla a=0.02 final theta = (7.1181, 10.2754), ||theta|| = 12.500, R_emp = 0.006



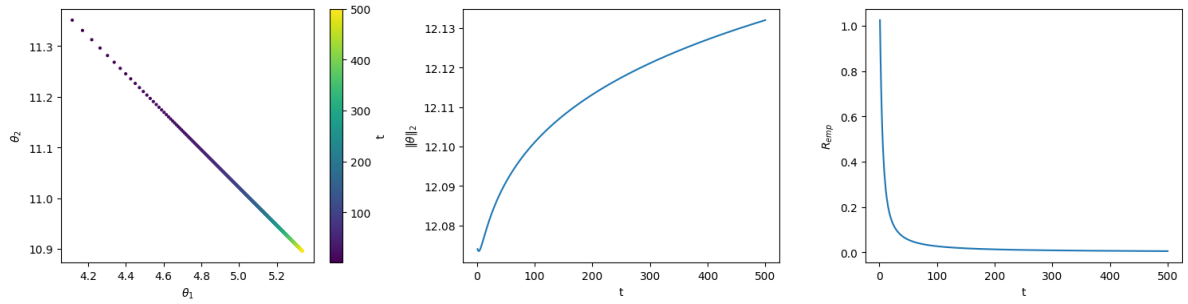
Python

```python
def grad(theta, lam):
    """Gradient of the L2-regularized logistic risk (note 2*lam*theta)."""
    theta = np.asarray(theta, dtype=float)
    sigma = 1 / (1 + np.exp(-(X @ theta)))
    return 2 * lam * theta + (sigma - y) @ X


def gd_step(theta, alpha, lam):
    return theta - alpha * grad(theta, lam)


def plot_fun(step_fn, lam, n_iters=500, theta0=np.zeros(2), label=""):
    """Run GD for n_iters; plot trace, ||theta||, R_emp; print final state."""
    theta = theta0.copy()
```

```
    thetas = np.empty((n_iters, 2))
    norms = np.empty(n_iters)
    fs = np.empty(n_iters)
    for i in range(n_iters):
        theta = step_fn(theta)
        thetas[i] = theta
        norms[i] = np.linalg.norm(theta)
        fs[i] = f(theta, lam)

    fig, axes = plt.subplots(1, 3, figsize=(15, 4))
    sc = axes[0].scatter(thetas[:, 0], thetas[:, 1],
                         c=np.arange(1, n_iters + 1), s=4, cmap="viridis")
    axes[0].set_xlabel(r"$\theta_1$"); axes[0].set_ylabel(r"$\theta_2$")
    fig.colorbar(sc, ax=axes[0], label="t")
    axes[1].plot(np.arange(1, n_iters + 1), norms)
    axes[1].set_xlabel("t"); axes[1].set_ylabel(r"$\|\theta\|_2$")
    axes[2].plot(np.arange(1, n_iters + 1), fs)
    axes[2].set_xlabel("t"); axes[2].set_ylabel(r"$R_{emp}$")
    plt.tight_layout()
    plt.show()

    print(f"{label:22} final theta = ({theta[0]:7.4f}, {theta[1]:7.4f}), "
          f"||theta|| = {norms[-1]:7.3f}, R_emp = {fs[-1]:.3f}")


## alpha = 0.01, lambda = 0 (vanilla GD, unregularized)
plot_fun(lambda theta: gd_step(theta, 0.01, 0), lam=0, label="vanilla a=0.01")
## alpha = 0.02, lambda = 0
plot_fun(lambda theta: gd_step(theta, 0.02, 0), lam=0, label="vanilla a=0.02")
```




```
vanilla a=0.01         final theta = ( 3.1035,  5.3043), ||theta|| =   6.146, R_emp = 0.011
```




```
vanilla a=0.02          final theta = ( 5.3362, 10.8954), ||theta|| =  12.132, R_emp = 0.005
```

Observation. Both step sizes drive the iterates outward indefinitely while the loss falls monotonically toward zero. The $\|\theta\|$ curves show a *steep early climb* followed by progressively slower growth. The iterates never plateau; they just creep outward more slowly as t grows.

The two step sizes differ mainly in the **terminal norm**: at iteration 500, $\|\theta\|$ at $\alpha = 0.02$ is roughly twice that at $\alpha = 0.01$ — the per-iteration motion scales with α, accumulated over a fixed number of steps.

Theoretically GD does not converge here because $\mathcal{R}_{\text{emp}}$ has no minimum (a). Empirically: the $\|\theta\|$ curve never plateaus, even when its rate of growth becomes very slow.

(e) Add L2 regularization

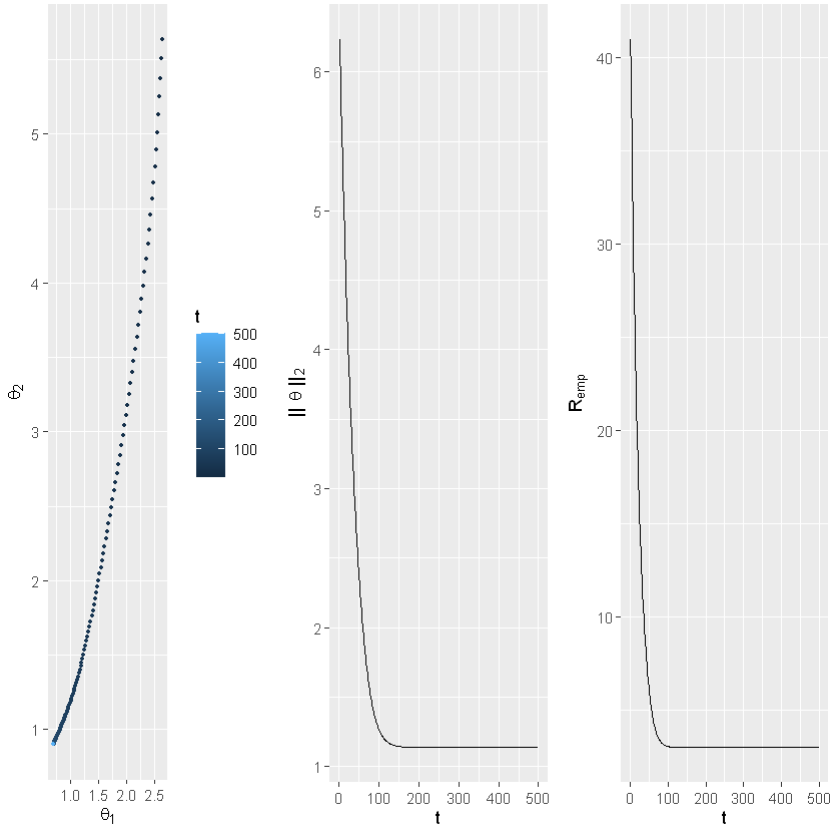
Repeat (d), but minimize $\lambda\|\theta\|^2 + \mathcal{R}_{\text{emp}}(\theta)$ with $\lambda = 1$. What changes?

Solution.

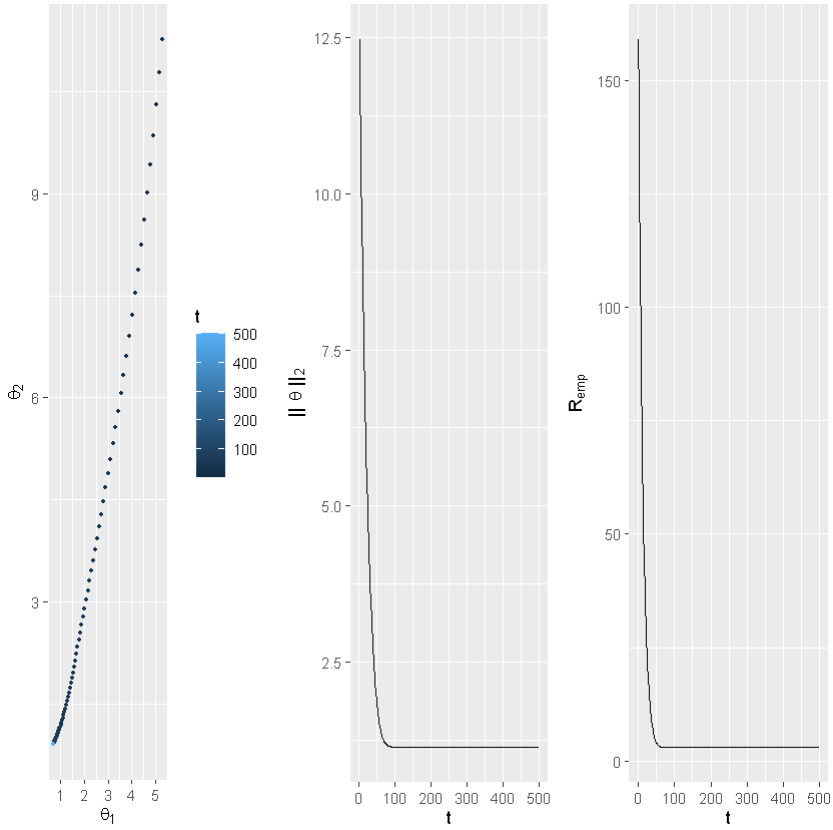
R

```r
## lambda = 1, alpha = 0.01
plot_fun(function(theta) gd_step(theta, 0.01, 1), lambda = 1,
         label = "reg a=0.01    ")
## lambda = 1, alpha = 0.02
plot_fun(function(theta) gd_step(theta, 0.02, 1), lambda = 1,
         label = "reg a=0.02    ")
```

```
reg a=0.01              final theta = ( 0.6985,  0.8986), ||theta|| =   1.138, R_emp = 2.985
```

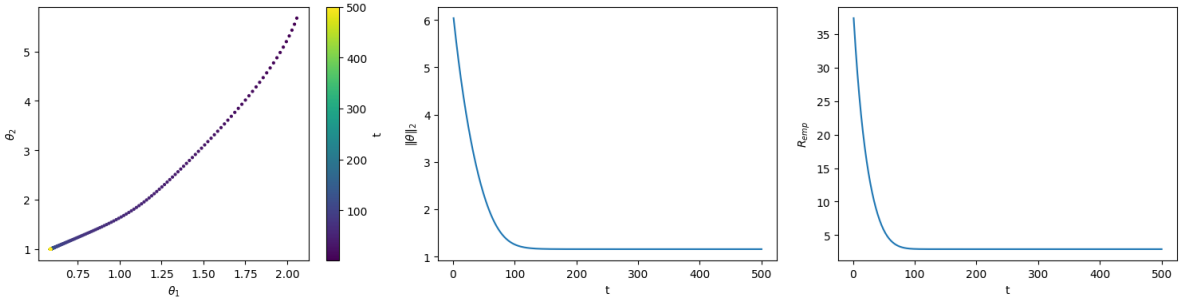



```
reg a=0.02              final theta = ( 0.6985,  0.8986), ||theta|| =   1.138, R_emp = 2.985
```

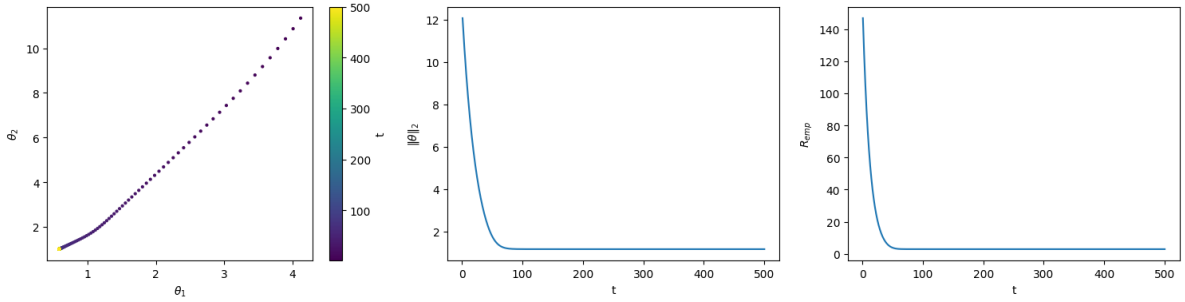


Python

```
## lambda = 1, alpha = 0.01
plot_fun(lambda theta: gd_step(theta, 0.01, 1), lam=1, label="reg a=0.01    ")
## lambda = 1, alpha = 0.02
plot_fun(lambda theta: gd_step(theta, 0.02, 1), lam=1, label="reg a=0.02    ")
```



```
reg a=0.01              final theta = ( 0.5860,  0.9966), ||theta|| =   1.156, R_emp = 2.907
```




```
reg a=0.02              final theta = ( 0.5860,  0.9966), ||theta|| =   1.156, R_emp = 2.907
```

What changes. The regularizer $\lambda\|\theta\|^2$ gives the objective a finite minimum. Instead of the iterates growing without bound as in (d), the $\|\theta\|$ curves now rise and then settle to a plateau — the signature of convergence — and both step sizes converge to the same minimum.

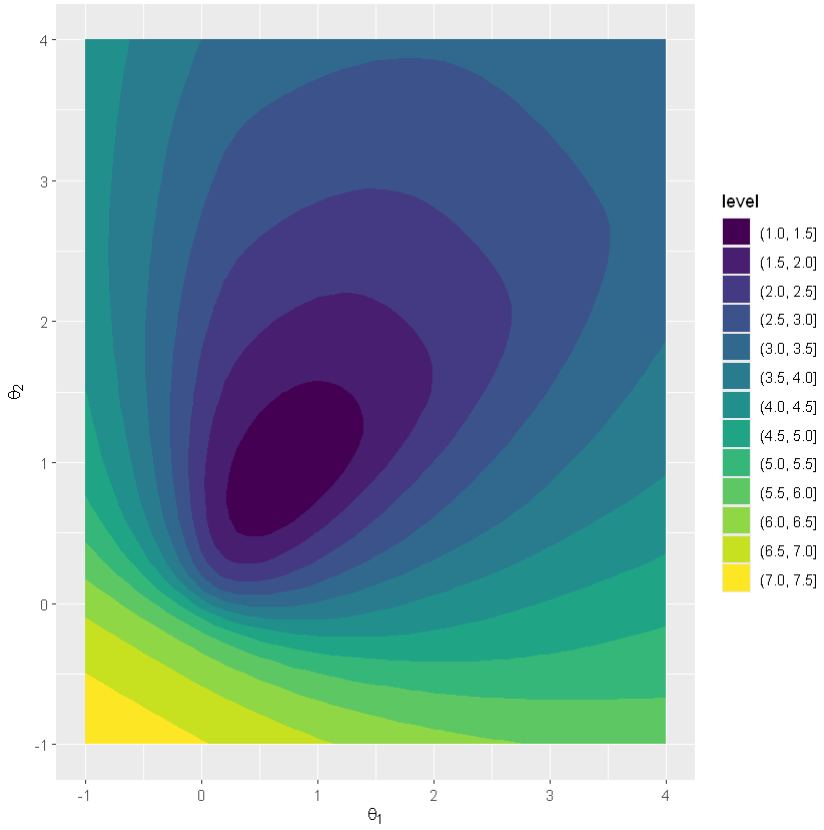
(f) Visualize the regularized empirical risk

Visualize the regularized empirical risk $\lambda\|\theta\|^2 + \mathcal{R}_{\text{emp}}(\theta)$ on $[-1, 4] \times [-1, 4]$ with $\lambda = 1$.

Solution.

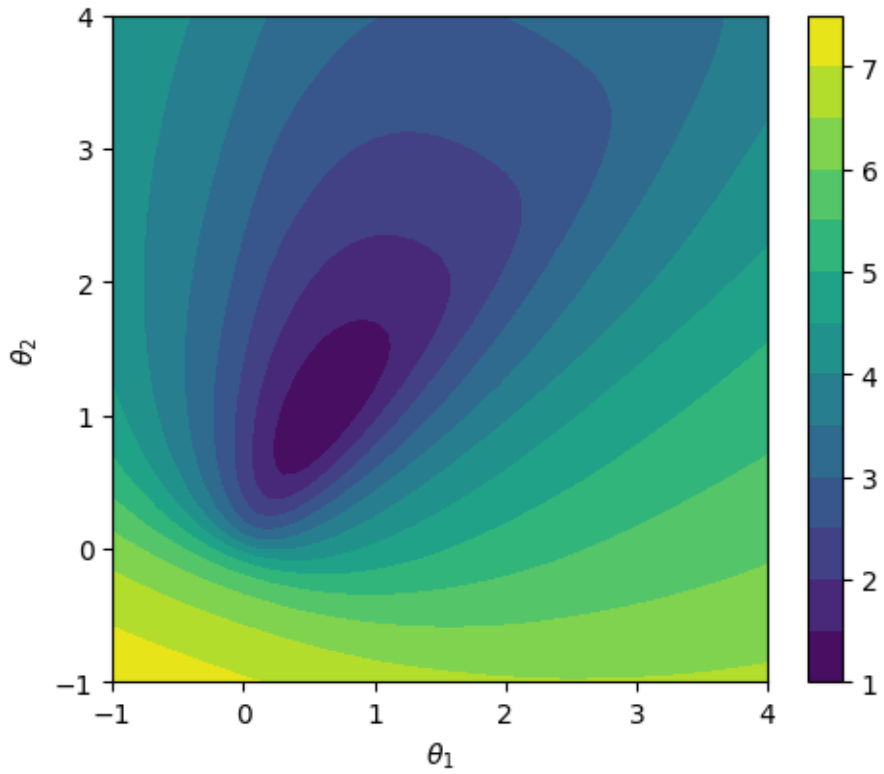
R

```r
fxx_reg <- log(apply(xx, 1, function(t) f(t, 1)))
df_grid_reg <- data.frame(xx, fxx = fxx_reg)

ggplot(df_grid_reg, aes(x = X1, y = X2, z = fxx)) +
  geom_contour_filled() +
  xlab(expression(theta[1])) +
  ylab(expression(theta[2]))
```



Python

```
Z_reg = np.array([[np.log(f(np.array([t1, t2]), 1)) for t1 in grid]
                  for t2 in grid])

fig, ax = plt.subplots(figsize=(5.5, 4.5))
cs = ax.contourf(T1, T2, Z_reg, levels=14)
ax.set_xlabel(r"$\theta_1$"); ax.set_ylabel(r"$\theta_2$")
fig.colorbar(cs)
plt.show()
```



Compare to (b): the regularized landscape has closed level sets around a finite minimum — a **bowl**, not a slide. This is what makes GD converge in (e). The minimum's location is biased toward the origin compared to the unregularized maximum-likelihood direction, by an amount depending on λ.

(g) Gradient descent with backtracking line search

Repeat (e) but with **backtracking line search**: at each step, propose $\theta_{\text{prop}} = \theta - \alpha \nabla f(\theta)$ and accept it if it satisfies the Armijo condition

$$f(\theta_{\text{prop}}) \leq f(\theta) - \gamma\, \alpha\, \|\nabla f(\theta)\|^2,$$

otherwise shrink $\alpha \leftarrow \tau\alpha$ and try again. Use $\gamma = 0.9$, $\tau = 0.5$.

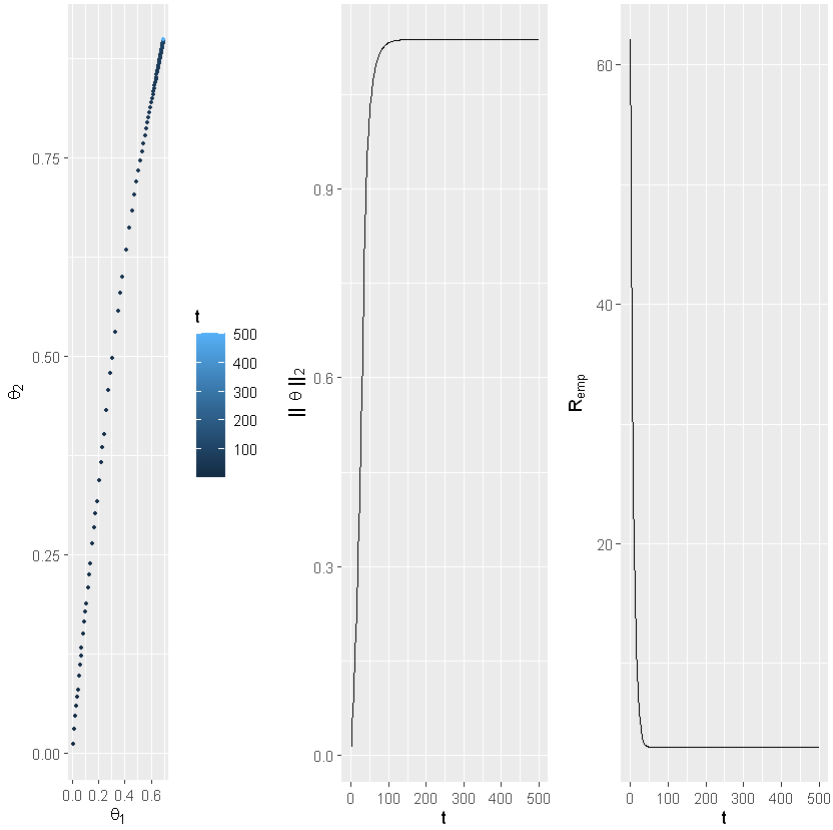
Note: the trace below visualizes from iteration $t = 1$ (after one backtracking step), not from the starting point $\theta^{[0]} = (0, 0)^\top$.

Solution.

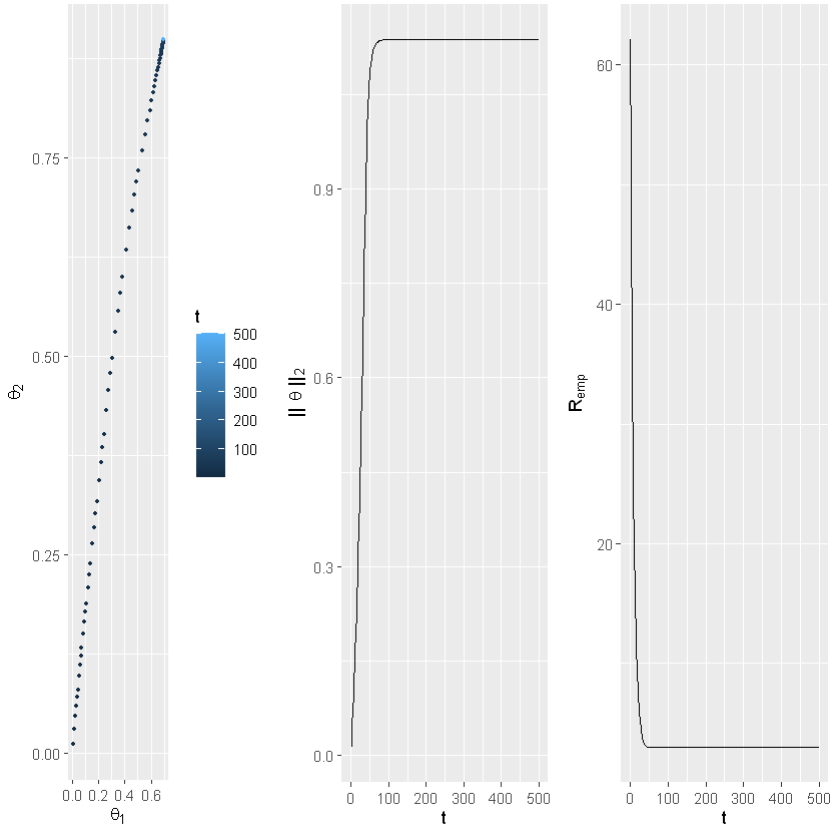
R

```r
gd_backtracking_step <- function(theta, alpha, gamma, tau, lambda) {
  f_theta <- f(theta, lambda)
  d_theta <- df_t(theta, lambda)[1, ]
  for (i in 1:1000) {
    theta_prop <- theta - alpha * d_theta
    if (f(theta_prop, lambda) <= f_theta - gamma * alpha * sum(d_theta^2)) {
      return(theta_prop)
    }
    alpha <- tau * alpha
  }
  stop("backtracking failed to find a descent step after 1000 shrinks; ",
       "check curvature / starting point / gamma=", gamma)
}

## lambda = 1, alpha_init = 0.01
result_bt <- plot_fun(
  function(theta) gd_backtracking_step(theta, 0.01, 0.9, 0.5, 1),
  lambda = 1, label = "backtrack a=0.01")
## lambda = 1, alpha_init = 0.02
plot_fun(
  function(theta) gd_backtracking_step(theta, 0.02, 0.9, 0.5, 1),
  lambda = 1, label = "backtrack a=0.02")
```

```
backtrack a=0.01       final theta = ( 0.6985,  0.8986), ||theta|| =   1.138, R_emp = 2.985
```



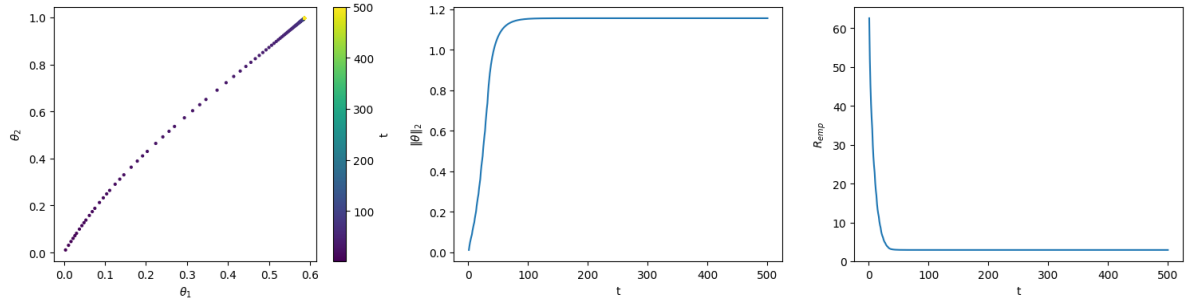

```
backtrack a=0.02       final theta = ( 0.6985,  0.8986), ||theta|| =   1.138, R_emp = 2.985
```



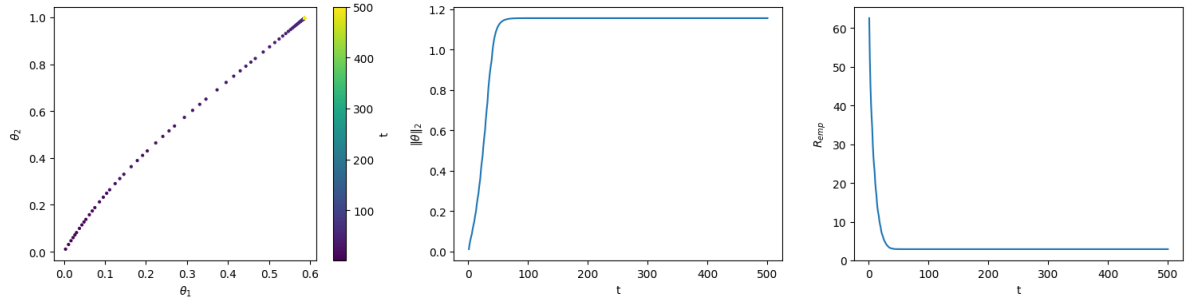
Python

```python
def gd_backtracking_step(theta, alpha, gamma, tau, lam, max_iter=1000):
    """Single GD step with Armijo backtracking. Shrinks alpha until accepted."""
    f_theta = f(theta, lam)
    d_theta = grad(theta, lam)
    grad_sq = d_theta @ d_theta
    for _ in range(max_iter):
        theta_prop = theta - alpha * d_theta
        if f(theta_prop, lam) <= f_theta - gamma * alpha * grad_sq:
            return theta_prop
        alpha *= tau
    raise RuntimeError(
        f"backtracking failed to find a descent step after {max_iter} shrinks; "
        f"check curvature / starting point / gamma={gamma}")
```

```
## lambda = 1, alpha_init = 0.01
plot_fun(
    lambda theta: gd_backtracking_step(theta, 0.01, 0.9, 0.5, 1),
    lam=1, label="backtrack a=0.01")
## lambda = 1, alpha_init = 0.02
plot_fun(
    lambda theta: gd_backtracking_step(theta, 0.02, 0.9, 0.5, 1),
    lam=1, label="backtrack a=0.02")
```




```
backtrack a=0.01        final theta = ( 0.5860,  0.9966), ||theta|| =   1.156, R_emp = 2.907
```




```
backtrack a=0.02        final theta = ( 0.5860,  0.9966), ||theta|| =   1.156, R_emp = 2.907
```

Observation. Backtracking adapts α per iteration: the step is automatically shortened when a proposed update would not produce a sufficient decrease. The loss curve is monotone non-increasing by construction (Armijo condition), and both initial α values converge to the same minimum found in (e).

Where this leaves us. This exercise traces a ladder of fixes:

- Vanilla GD on separable data $\rightarrow$ diverges (a, d).

- L2 regularization → finite minimum (e).
- Backtracking → adaptive step that does not depend on the user picking the right α up front (g).

The next stage is **direction**: even with a well-chosen step, the local-gradient direction is suboptimal on poorly-conditioned problems — when the level sets are elongated, the negative gradient points mostly across the valley rather than along it, so the iterates zig-zag and progress slowly. Exercise 7 introduces *momentum* to break that limitation.